Ejercicio 2 (1 punto)

Sin definir otras funciones adicionales, implementar una función en lisp llamada añadirValor que, dada una lista que pudiera ser anidada y un valor, añada el valor a todos los elementos incluidos en la lista (anidados o no)

(añadirValor '(1 (2 3) (4 2) 3) 2) -> (3 (4 5) (6 4) 5)

Ejercicio 3 (1,5 puntos)

Sin definir otras funciones adicionales, implementar una función en lisp llamada aplicarFunciones que, dadas dos funciones unarias y un número arbitrario de valores numéricos, devuelva una lista de pares de valores, donde cada par se corresponda con la aplicación de cada una de las dos funciones sobre cada uno de los argumentos numéricos.

(aplicarFunciones #'1+ #'1+ 2 3 1) -> ((3 3) (4 4) (2 2))

(aplicarFunciones #'1+ #'1- 2 3 6 1) -> ((3 1) (4 2) (7 5) (2 0))

(aplicarFunciones (lambda(x) (* 2 x)) #'1- 6) -> ((12 5))

Ejercicio 4 (2 puntos)

Sin definir otras funciones adicionales, implementar una función en lisp llamada computarFunciones que, dada una lista de valores numéricos y un número arbitrario de funciones unarias, devuelva una nueva lista donde cada elemento sea una lista con los resultados de aplicar una de las funciones sobre cada uno de los elementos de lista pasada como parámetro.

(computarValores '(1 3 4) #'1+ #'1+) -> ((2 4 5) (2 4 5))

(computarValores '(1 3 4) (lambda(x) (* 2 x)) #'1+ (lambda(x) (- x 1))) -> ((2 6 8) (2 4 5) (0 2 3))

Ejercicio 5 (2 puntos)

Sin definir otras funciones adicionales, implementar una función en lisp llamada separarPositivosYNegativos que, dado un número arbitrario de argumentos, devuelva dos listas, la primera con los elementos positivos y la segunda con los elementos negativos.

(separarPositivosYNegativos 1 -1 0 -2 -8 6 3) -> ((1 6 3) (-1 -2 8))